Project Search Statistical Initial Results

Getting statistical results for my agent in my environment was quite a journey. With my initial search agent being focused on the defender going up against a random agent, it was clear that without any kind of strategy or knowledge to take specific actions, it was extremely unlikely for the random agent attacker to ever beat the defender. I ran hundreds of games, and indeed, the defender won 100% of the time.

I expected this to be the case, however, as the objective of the defender is very simple – get the king to any of the edges of the board. However, an attacker has to have 2 of its pieces in opposing spaces adjacent to the king in order to win. So, instead of just having the defender go up against a completely random agent, I decided that I needed to implement an attacker agent, and then introduce some aspect of randomness into one or both of the agents in order to make it less deterministic.

For the attacker agent, I once again used the minimax algorithm with alpha beta pruning. However, the actual evaluation function was much more complicated. It took a few things into consideration:

- How many defenders remain afterwards, rewarding having fewer defenders.
- The king's distance to the center, trying to keep the king as far from the edges as possible
- The attacker's proximity to the king, prioritizing being near the king to hopefully be able to capture it
- How many defenders it could possibly capture from that position on its next turn, favoring higher advantage spots
- How many of the king's escape routes are blocked from that position

Having implemented this, in 20 games the attacker agent would win about 75% of the games with 28 moves. But, then I realized a major issue.

Because both the defender agent and attacker agent used the mini-max algorithm with alpha beta pruning, their moves were completely deterministic. I noticed this when I manually looked through the games and saw that they were making the exact same moves and ending up with the exact same result every time. So I had to go back to the drawing board a little, and come up with a way to add some randomness.

I knew that I had to give the randomness to only one of the 2 agents, so I decided it might be a little better to have the attacker get the randomness. This is because, the way that I went about adding randomness was to create an array of the "best moves". Anytime a move had a higher score than the current highest score, it would add this move to the array. The select_action method then selects one of these moves at random to be the move it plays.

The randomness definitely worked to make the games different from one another which was good. But there was something definitely a little alarming, which was that despite all of the knowledge and strategy the attacker was utilizing, the defender was still winning 75% of the games when I ran it 20 times. I thought there was no way this should be correct, and I suspected it was due to the randomness I added.

Because of how the best moves array randomness is handled, every single one of the 'best moves' has the same chance of being picked, regardless of whether it's actually a good move or if it was simply better than what had been found at the very start, but still objectively pretty bad.

So, I came up with the idea to give the best moves weights based on how high the score is. I did some research on the topic, and that led me down a rabbit hole of neural networking stuff that was out of scope for this project, but I did discover something that would be useful to me, which was softmax.

Using softmax, I take the list of best moves, and they get exponentiated. This makes it so that higher scores receive disproportionately more weight than lower scores. They're then normalized so that they add up to 1. With this implementation, we're avoiding deterministic behavior while still biasing towards good moves. By having temperature scaling, much like with the policy search algorithm, I can make the attacker more exploratory and random if I choose to.

This ended up working very well. All of the games were different from one another, if ever so slightly. If they did have a lot of similarities, it's most likely just because of how simply the defense agent is at this current stage. After running 20 games, I ended up with:

- 18 Attacker wins
- 2 Defender wins
- 32.1 Moves per game on average

The attacker was now a lot stronger than the defender of course, considering how simple the defender agent's implementation is compared to the attacker. This means there was now a strong opponent with some good variation, and I set my sights on the goal of making a defender agent that's strong enough to go against it.

From here I created two new versions of the defender agent, each one improving on the previous version immensely. For the second version, I made the evaluation function take a lot more into consideration. Rather than only trying to get scenarios where the defender tries to move the king as close to the edge as possible, disregarding all other strategy, it also does the following:

- Gets penalized if the king remains in the castle for too long
- Rewarded for having more defenders near the king
- Rewarded for the king having high mobility (a lot of spaces it can move to)
- Rewarded for the number of defenders remaining
- Penalized quite heavily for how many immediate threats the king has
- Rewarded quite heavily for how many attackers they have the potential to capture
- Small reward for having high mobility on all other defender pieces
- Penalty for revisiting the same states

With these strategies implemented, the defender was winning quite a bit more, but the games were still heavily in favor of the attacker. Where previously the defender was winning 10% of its games, it was now winning 40%. This is a decent increase, but I wasn't quite satisfied with this result.

This led me to creating the third and greatest agent in the project. For the third version of the defender agent, I decided to actually research game winning strategies for playing as a defender in Tablut. The strategy that I discovered revolved around the idea of following a set of priorities anytime you're making a move. It included the following:

1. Never move the king

    a. Even though in order to win, you need to move the king to the edge, the board is best defended while the king is in the middle (the castle). We can only move the king to the edge after we've removed enough enemy pawns to ensure victory.
2. If you can take an enemy pawn, do it
    a. Anytime you have the opportunity to take an enemy pawn, you need to do it. The enemy pawns outnumber the defenders by a few, but if you can stop them from ever getting into an advantageous position by taking them out, it's important to do so.
3. If your pawns are out of position compared to where they started and you can't take an enemy pawn, move your pawns back to where they started.
    a. The starting position is actually incredibly powerful, because you have full control over the center 5x5 tiles of the board. You can pretty much immediately take an enemy pawn if it gets moved next to yours, no matter which tile it gets moved to.
4. If you can't take an enemy pawn, and you're in the starting position, make the safest move possible.
    a. This is basically just to stall for time and force the enemy to move into another position where they can be captured. On the next turn, if you still can't take an enemy pawn, you can just simply move the defender back into the starting position.

With this in mind, I started updating the evaluation function to be more aligned with this strategy. I also updated the select_action method to check to see if the conditions are met, and sorted them by priority before doing the evaluation functions to get the best move. It's also extremely important to note that I also increased the depth of the minimax algorithm to 3 instead of 2, because it wasn't quite looking deep enough.

The results were very effective, with the defenders now winning 90% of their games with an average length of 26.1 moves. So not only were the defenders winning a lot more often, but the average length of the games also went down. Finally, I had an agent that is able to regularly beat a pretty tough opponent, and this is where I finally felt satisfied with the results. Going up against a random agent wasn't nearly enough of a challenge to create an agent around. Creating two separate agents, one as attacker and one as defender, was a lot more work and it was a lot more difficult, but it paid off in the end.

Final Initial Results Table

Defender Agent	Attacker Agent	Games Played	Defender Win %	Attacker Win %	Avg Game Length	Notes
Random	Random	10	70%	30%	71.5	Baseline
Version 1	Random	10	100%	0%	11.6	
Version 1	Version 1	10	10%	90%	33.8	New Baseline
Version 2	Version 1	10	40%	60%	32.8	Slightly improved
Version 3	Version 1	10	90%	10%	26.1	Great Defensive Strategy

In conclusion, simply going up against random agents in this case was not nearly good enough. Because of the two varying playstyles of the attackers and defenders, it was necessary to build them both up to a point where we can have a real challenge. I hope to simulate more games for the final statistical results, but beyond that there isn't much left to do for my project.